iSCSI×ZFS
ディスクレスPCで瞬間復元サンドボックス環境の構築

IN3A 辻 昂志朗

研究の背景

なぜこのシステムが必要なのか

マルウェア検証や破壊的テストを日常的に行う開発・研究現場が抱える課題

OSが破壊されるたびに再構築が必要になり、時間とストレージ寿命を消耗する。さらに物理ディスクが残る限り、情報漏洩のリスクも消えない。

復旧コストの増大

クリーンインストールに膨大な時間がかかる

物理ストレージの摩耗

SSDの書き込み寿命（TBW）を急速に消費

データ流出リスク

物理ディスク経由でのマルウェア・機密漏洩

研究の目的

システム全体構成（アーキテクチャ）

「物理ディスクを持たない」×「1秒で初期状態に戻せる」実験環境を、低コストで構築する

クライアント側（実機PC）

※物理ディスク未搭載

1. 初期起動層

通常のPXE (LANカード)
サーバーから ipxe.efi を取得

2. 拡張ブート層

iPXE環境。iSCSIプロトコルを
有効化しLUN 0を直接マウント

3. ゲストOS

Windows（完全ディスクレス起動）
ネットワーク越しのLUNを内蔵ディスクと誤認識

iSCSI →

サーバー側（Lubuntu）

物理PC + 外付けSSD

ストレージ層　ZFSプール (tank)

ZVOL: win10_disk（70GB 仮想ブロック）

データ配信・制御層　(dnsmasq & targetcli)

DHCP/TFTP：ipxe.efi を配信
iSCSI：ZVOLを LUN 0 として公開（デモモード）

コア技術 ①

ネイティブ iSCSI ブートとは

仕組み

電源投入直後、BIOS（またはKVM）が直接サーバーへ接続し、「ネットワークの先のディスクを、物理SATAで繋がった内蔵ディスクである」とOSに錯覚させる。

メモリ一括展開が不要なため相性問題が起きない

キーワード

iBFT

iSCSI Boot Firmware Table。BIOSがOSへ「このディスクはネットワーク越し」という情報を引き継ぎ、起動途中でも自発的にネットワークドライバを読み込ませる。

TCP 3260

iSCSIプロトコルの通信ポート

接続を切断せず安定動作 → 完全ディスクレスを実現

試行錯誤

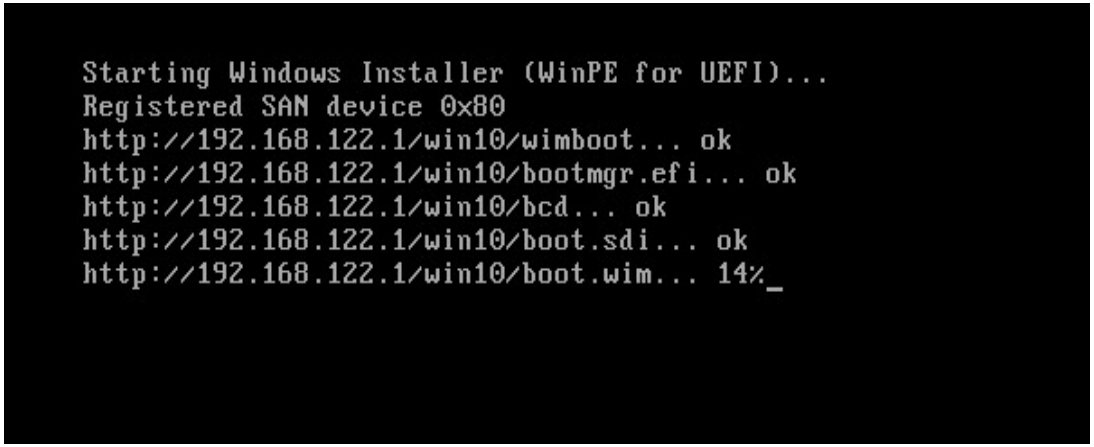
従来手法「iPXE」での失敗と限界

HTTP経由でboot.wim（約4GB）を全てメモリに展開するwimboot方式を試みたが、2つの壁に阻まれた

BIOS

2GBメモリの壁

古いBIOS（SeaBIOSなど）はメモリ空間に制限があり、4GBのファイル読み込みでキャパオーバー。「Loading files...」の表示のままクラッシュする。



結論：iPXE（メモリ上への展開）方式は根本的に不採用とし、方式を転換

試行錯誤

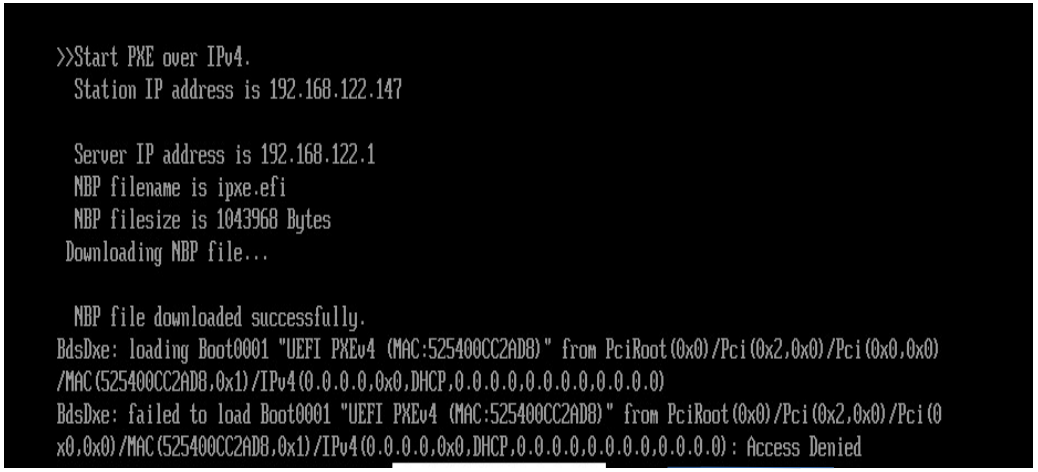
従来手法「iPXE」での失敗と限界

HTTP経由でboot.wim（約4GB）を全てメモリに展開するwimboot方式を試みたが、2つの壁に阻まれた

UEFI

セキュリティ保護の壁

最近のノートPCのUEFIはセキュリティが厳しく、wimbootが強引にメモリを書き換えようとした瞬間に不正アクセスとみなされ起動ができない



結論：iPXE（メモリ上への展開）方式は根本的に不採用とし、方式を転換

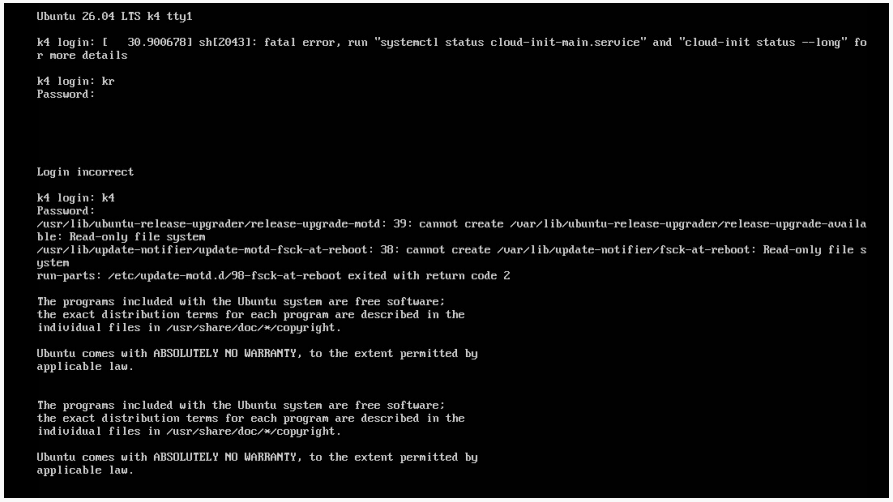
試行錯誤

従来手法「iPXE」での失敗と限界

他に試したパターン



インストールだけローカルで済ませて起動したパターン



Linux(Ubuntu)で試したパターン(検証途中)

結論：iPXE（メモリ上への展開）方式は根本的に不採用とし、方式を転換

解決策

ネイティブ iSCSI ブートの優位性

メモリ不足や相性問題を回避

巨大ファイルをメモリへ一括ダウンロードする必要がないため、BIOSのメモリ制限にも最新UEFIのセキュリティ保護にも抵触しない

iBFTによる自動ドライバ読込

BIOSがOSへiBFTを引き継ぎ、Windowsが起動途中で自発的にネットワークドライバを読み込む

通信断絶なしで安定動作

起動シーケンス全体を通してiSCSIセッションが維持され、通信を切断することなく安定して動作し続ける

コア技術 ②

ZFS による瞬間復元（ロールバック）

コピー・オン・ライト（CoW）

サーバーが公開する仮想ディスクの正体は、ZFS上のブロックデバイス（ZVOL）。ZFSはコピー・オン・ライト方式でデータの変更履歴を極めて効率的に管理する。

スナップショット作成

テスト前の状態を保存。処理は数秒、容量消費は0バイト。

zfs rollback 1行

破壊テスト後、数秒でバイト単位に完全復元する。

復元にかかる時間

1〜2秒

通常のイメージリストアは数十分。バイト単位の完全復元がわずか1~2秒で完了する。

何度でも危険なテストを繰り返せるサンドボックス化を実現する。

運用フロー

高速復元サンドボックスの運用フロー

1

スナップショット作成

綺麗な状態のWindowsを保存。処理は1秒、容量消費は0バイト

2

破壊的テストの実行

ランサムウェアの全暗号化やSystem32削除など、危険な検証を実行

3

瞬間ロールバック

zfs rollback を1行実行。1秒で無傷な状態にバイト単位で復元

PCをシャットダウンしてサーバー側で実行するだけ。クライアントには何も残らない。

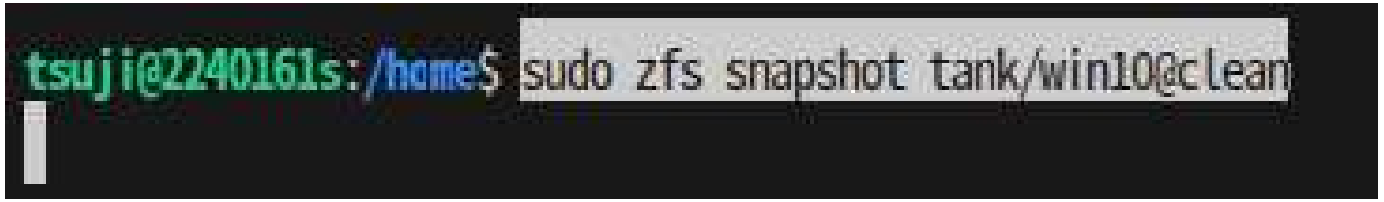
運用フロー

高速復元サンドボックスの運用フロー

1

スナップショット作成

綺麗な状態のWindowsを保存。処理は1秒、容量消費は0バイト



スナップショット：sudo zfs snapshot [zfsパーティションのパス]

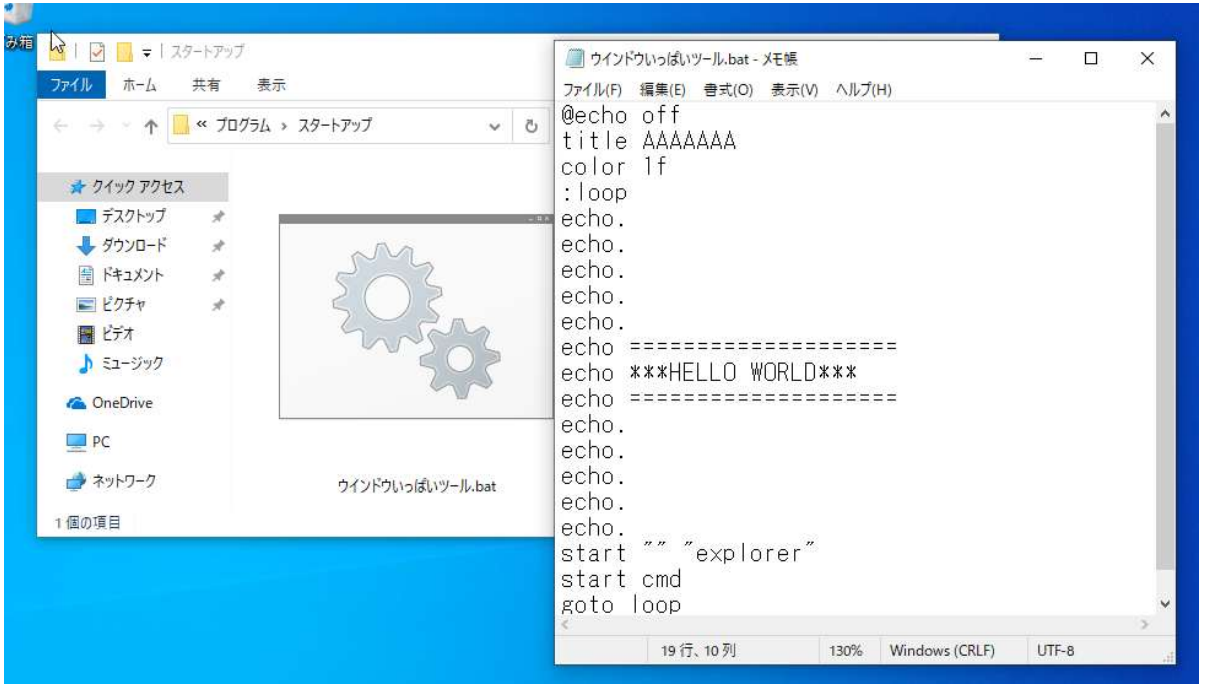
運用フロー

高速復元サンドボックスの運用フロー

2

破壊的テストの実行

ランサムウェアの全暗号化やSystem32削除など、危険な検証を実行



スナップショット：sudo zfs snapshot [zfsパーティションのパス]

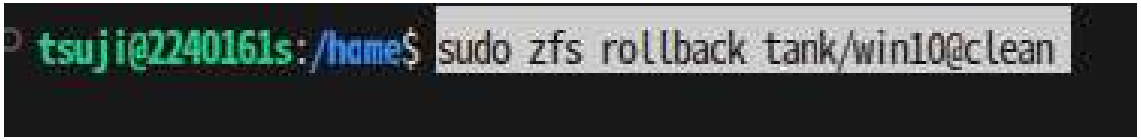
運用フロー

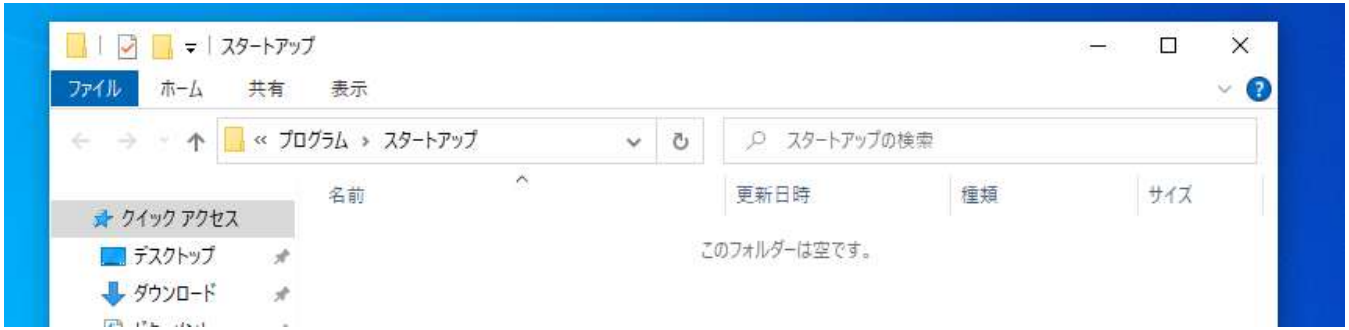
高速復元サンドボックスの運用フロー

3

瞬間ロールバック

zfs rollback を1行実行。1秒で無傷な状態にバイト単位で復元





ロールバック：sudo zfs rollback [zfsパーティションのパス]

導入効果 ①

安全性の向上



クライアントPC側に物理ディスクが存在しないため、物理的な盗難やデータの持ち出しによる情報漏洩リスクを低減できる。

解決

マルウェア本体

テスト後は自動的に消滅。実機ディスクに一切残留しない

解決

機密データ

サーバー側のZFSでのみ管理。クライアント持ち出しは不可能

解決

攻撃対象領域

ディスクレスにより物理層からの侵害経路が根本的に消える

導入効果 ②

効率・低コスト・スケーラビリティ

テストサイクル

大幅短縮

1秒でリセットできるため、解析・検証サイクルを劇的に高速化

導入コスト

OSSのみ

Deep FreezeやCitrix等の高額製品なしに実用的な環境を実現

スケーラビリティ

数十台規模

ZFSのクローン機能で、1つのイメージから一瞬で複製可能

まとめ

オープンソース技術だけで実現する新しい実験基盤

従来の課題

- 課題 OS破壊のたびに再インストールが必要
- 課題 SSDの書き込み寿命を消耗
- 課題 物理ディスク経由の情報漏洩リスク
- 課題 復元に数十分かかる

本システムによる解決

- 解決 完全ディスクレスで再インストール不要
- 解決 書き込みはサーバー側ZFSに一元化
- 解決 物理ディスク自体が存在せず情報漏洩リスクを低減
- 解決 zfs rollback で約1〜2秒で復元

ご清聴ありがとうございました